

Client Integration Requirements

Tech Support Copilot — External System Integration Specification

Version: 1.0

Date: 2026-05-02

Audience: Client engineering teams onboarding their own infrastructure into the Tech Support Copilot platform.

Overview

The Tech Support Copilot is an agentic AI platform that orchestrates a plan → act → observe loop to assist live support agents. It connects to five categories of external systems via a pluggable tool layer. Each category below describes what the platform expects from the client's corresponding system, including the exact API contract, required fields, error behavior, and security posture.

The integration point is `tools/` — each class in that directory wraps one external API call. A client replaces the mock implementations with live HTTP clients pointing at their own systems. No orchestrator or LLM code changes are required.

1. CRM / Customer Data System

Tools: `LookupCustomer`, `GetTicketHistory`

Source file: `tools/crm_tools.py`

1.1 `GET /customers/{customer_id}` — Customer Profile

The platform calls this once at session start to establish customer identity.

Required response fields:

Field	Type	Description
customer_id	string	Stable unique identifier (e.g., CUST001)
name	string	Full display name
email	string	Primary contact email
phone	string	E.164-formatted phone number
account_status	enum	active suspended cancelled
service_tier	string	Human-readable plan name (e.g., Gigabit 500)
account_since	string	ISO 8601 date (YYYY-MM-DD)
zip_code	string	5-digit US ZIP (used for outage geo-lookup)
region	string	Internal region slug (e.g., northeast)
modem_mac	string	MAC address of the primary CPE device
equipment.modem	string	Modem make/model string
equipment.router	string	Router make/model string (may be null)

Behavior requirements:

- Must return 404 with { "error": "Customer not found" } for unknown IDs.
- Response time: ≤ 500 ms (p99). The agentic loop counts this call against its per-turn latency budget.
- Customer ID format must be consistent with the IDs used in tickets and network systems — the agent uses the same ID across all tool calls in a session.

1.2 GET /customers/{customer_id}/tickets — Ticket History

Required response fields (array of ticket objects):

Field	Type	Description
ticket_id	string	Globally unique ticket reference
opened	string	ISO 8601 date the ticket was opened
closed	string null	ISO 8601 date closed, or null if open
issue	string	One-sentence description of the reported problem
status	enum	open resolved escalated
resolution	string null	Free-text summary of how the issue was resolved

Behavior requirements:

- Return results newest-first.
- The agent reads up to the last 10 tickets; include a `limit` query parameter.
- An empty array `[]` is valid for new customers.

2. Network Diagnostics System

Tools: `PingCustomerModem`, `CheckModemStatus`, `RunTraceroute`

Source file: `tools/network_tools.py`

The network diagnostic layer is the most latency-sensitive surface. The agent calls these tools to distinguish customer-side issues from ISP-side issues before proposing any action.

2.1 `POST /diagnostics/ping` — Ping CPE

Request body:

```
{ "customer_id": "CUST001" }
```

Required response fields:

Field	Type	Description
<code>customer_id</code>	<code>string</code>	Echo of the request ID
<code>latency_ms</code>	<code>number</code>	Average round-trip latency in milliseconds
<code>packet_loss_pct</code>	<code>number</code>	Packet loss percentage <code>0.0</code> – <code>100.0</code>
<code>status</code>	<code>enum</code>	<code>healthy</code> <code>degraded</code> <code>unreachable</code>
<code>hops_to_modem</code>	<code>integer</code>	Number of network hops
<code>note</code>	<code>string</code>	Human-readable summary for the LLM

Thresholds the agent reasons over:

- `packet_loss_pct > 5` → agent will flag signal issue
- `latency_ms > 150` → agent will flag congestion or line issue
- `status == "unreachable"` → agent escalates to dispatch

2.2 `GET /diagnostics/modem/{customer_id}` — Modem Status

Required response fields:

Field	Type	Description
online	boolean	Whether the modem is currently registered
uptime_hours	number	Hours since last reboot
restarts_last_24h	integer	Reboot count in rolling 24-hour window
downstream_dbmv	number	Downstream power level in dBmV (nominal: -7 to +7)
upstream_dbmv	number	Upstream power level in dBmV (nominal: 38-48)
snr_db	number	Signal-to-noise ratio in dB (target: ≥ 30)
status	enum	healthy degraded offline

Note: Signal level thresholds are used directly by the LLM to diagnose whether a field technician dispatch is warranted. Values outside nominal ranges must be faithfully reported — do not normalize.

2.3 POST /diagnostics/traceroute — Traceroute

Required response fields:

Field	Type	Description
hops	array	Ordered list of hop objects (see below)
destination_reached	boolean	Whether the final hop reached the target
total_hops	integer	Count of hops in the array

Hop object:

```
{
  "hop": 1,
  "ip": "192.168.1.1",
  "hostname": "gateway.local",
  "latency_ms": 2
}
```

3. Outage Detection System

Tools: CheckISPOutage , CheckDowndetector

Source file: tools/outage_tools.py

The agent checks for active outages **before** initiating any per-customer diagnostic. If an outage is active, it skips diagnostics and drafts a holding message for the customer.

3.1 GET /outages/active — ISP Outage Status

Query parameters: `customer_id`, `zip_code` (optional override)

Required response fields:

Field	Type	Description
<code>outage_active</code>	<code>boolean</code>	Whether a known outage is affecting the area
<code>outage_id</code>	<code>string</code> <code>null</code>	Internal outage ticket reference
<code>affected_zips</code>	<code>array[string]</code>	List of affected ZIP codes
<code>eta_resolution</code>	<code>string</code> <code>null</code>	ISO 8601 datetime of estimated restoration
<code>checked_zip</code>	<code>string</code>	ZIP actually checked (resolved from customer record if not overridden)
<code>last_updated</code>	<code>string</code>	ISO 8601 datetime of last status refresh
<code>note</code>	<code>string</code>	Free-text status summary

3.2 GET /outages/downdetector — Crowd-Sourced Report Spike

Query parameters: `service` (e.g., `internet`), `region`, `zip_code`

Required response fields:

Field	Type	Description
<code>spike_detected</code>	<code>boolean</code>	Whether report volume significantly exceeds baseline
<code>report_count_last_hour</code>	<code>integer</code>	Absolute report count in the last 60 minutes
<code>baseline_hourly_avg</code>	<code>integer</code>	Normal hourly report rate for reference
<code>spike_threshold_multiplier</code>	<code>number</code>	The multiplier at which a spike is declared (e.g., <code>3.0</code>)
<code>region</code>	<code>string</code>	Region the data covers
<code>checked_at</code>	<code>string</code>	ISO 8601 timestamp

4. Modem Management / Remote Action Platform

Tools: `RebootModemRemotely`, `SendSMSToCustomer`

Source file: `tools/mutating_tools.py`

Risk tier: `HIGH_RISK_MUTATING` — these tools require explicit human approval before execution. The platform will **never** auto-execute these.

4.1 POST /devices/{customer_id}/reboot — Remote Modem Reboot

Request body:

```
{
  "customer_id": "CUST001",
  "reason": "High packet loss and signal degradation. Customer consented to reboot."
}
```

Required response fields:

Field	Type	Description
reboot_initiated	boolean	Confirmation the command was accepted
reboot_initiated_at	string	ISO 8601 timestamp
estimated_restore_at	string	ISO 8601 expected restore time
status	enum	rebooting failed not_supported
note	string	Human-readable status

Requirements:

- The `reason` field must be persisted in the client's audit log.
- Idempotency: if a reboot is already in progress for this device, return the existing command status rather than issuing a second reboot.
- If the device does not support remote reboot (e.g., customer-owned modem), return `status: not_supported` with `HTTP 200` — do **not** return an HTTP error. The agent handles this gracefully.

4.2 POST /notifications/sms — Outbound SMS

Request body:

```
{
  "customer_id": "CUST001",
  "message": "Your modem has been reset. Service should restore within 2 minutes. Ref: TKT-XYZ."
}
```

Requirements:

- Message body enforces a **160-character limit** before sending (the platform pre-validates, but the API must also reject over-length messages with `400`).
 - The platform uses the phone number stored in the CRM. The SMS API should resolve it server-side from `customer_id` — do not pass the phone number from the agent (avoids PII leakage in logs).
 - Return `{ "message_sid": "<id>", "status": "queued" }` on success.
-

5. Knowledge Base

Tool: SearchKB

Source file: tools/kb_tools.py

The current implementation scans Markdown files from data/kb/. A client must replace this with a call to their own KB or documentation search system.

5.1 GET /kb/search — Full-Text KB Search

Query parameters: q (free-text query string), limit (default 3, max 8)

Required response format (array of result objects):

Field	Type	Description
doc_id	string	Stable document identifier (used for citation)
title	string	Article title
snippet	string	200–400 character excerpt most relevant to the query
score	number	Relevance score 0.0 – 1.0 (used for ranking display)
url	string null	Deep-link to the article in the client’s KB portal (shown to the agent)

Requirements:

- The agent passes a **free-text natural language query**, not a structured keyword list. The KB system must support full-text or semantic search.
- Results must be **ranked by relevance** — the agent takes the top result as the primary citation.
- An empty array is valid; the agent will fall back to reasoning without a KB citation.
- The following article categories must be covered for ISP support scenarios:

Category	Example topics
Connectivity	modem reboot steps, DNS issues, ethernet vs. WiFi
Equipment	device compatibility, firmware update procedures
Account	account suspension reasons, password reset, billing
Outage	outage communication templates, ETAs
Performance	slow speeds, peak-hour congestion, WiFi channel selection

6. Authentication & Security

All API endpoints called by the tool layer must conform to:

Requirement	Specification
Transport	HTTPS only. TLS 1.2 minimum, TLS 1.3 preferred.
Authentication	OAuth 2.0 Client Credentials flow (<code>client_id</code> + <code>client_secret</code> → bearer token). Token endpoint must support <code>expires_in</code> .
Token storage	Tokens are loaded from environment variables at startup — never hard-coded. Configure via <code>.env</code> : <code>CRM_CLIENT_ID</code> , <code>CRM_CLIENT_SECRET</code> , <code>CRM_TOKEN_URL</code> , etc.
Scopes	Read-only tools require a <code>read</code> scope. Mutating tools (<code>reboot</code> , <code>sms</code>) require a separate <code>write</code> scope that must be explicitly provisioned.
Audit logging	Every call to a mutating endpoint must include an <code>X-Agent-Session-Id</code> header (the session UUID) so client systems can correlate actions back to agent sessions.
PII	Customer names, emails, and phone numbers are fetched but must not be logged in plaintext by the agent's observability tracer. Mask PII fields in log output.
IP allowlisting	Provide the deployment IP range so the client can allowlist the agent's egress.

7. Performance SLAs

The agentic loop has a hard cap of 6 iterations per user turn. Slow APIs directly degrade the perceived response time of the copilot. Clients must target:

System	p95 Latency	p99 Latency
CRM profile lookup	≤ 300 ms	≤ 500 ms
CRM ticket history	≤ 400 ms	≤ 600 ms
Network ping/modem status	≤ 500 ms	≤ 1000 ms
Outage status	≤ 300 ms	≤ 500 ms
KB search	≤ 600 ms	≤ 1200 ms
Remote modem reboot	≤ 2000 ms	≤ 3000 ms
SMS dispatch	≤ 1000 ms	≤ 2000 ms

8. Error Contract

All endpoints must follow this uniform error envelope so the tool layer can surface meaningful messages to the agent:

```
{
  "error": "short_snake_case_code",
  "message": "Human-readable description.",
  "retryable": true
}
```

HTTP Status	Meaning	Agent behavior
200	Success	Parse response normally
400	Bad request (invalid params)	Log error, do not retry, surface to rep
401	Auth failure	Log error, attempt token refresh once
403	Insufficient scope	Log error, surface "action not permitted" to rep
404	Resource not found	Return <code>success: false</code> to orchestrator
429	Rate limited	Retry with exponential backoff (max 2 retries)
500 / 503	Server error	Retry once; if still failing, simulate tool failure and continue

The `retryable` field must be set to `true` for `429` and `503` only.

9. Mock Data Replacement

The following files are the reference data contracts. Replace them with live API adapters:

Mock file	Replace with
<code>data/mock_customers.json</code>	<code>CRM_BASE_URL</code> HTTP adapter in <code>tools/crm_tools.py</code>
<code>data/kb/*.md</code>	<code>KB_BASE_URL</code> HTTP adapter in <code>tools/kb_tools.py</code>
Hardcoded network responses in <code>tools/network_tools.py</code>	<code>NETWORK_BASE_URL</code> HTTP adapter
Hardcoded outage data in <code>tools/outage_tools.py</code>	<code>OUTAGE_BASE_URL</code> HTTP adapter
Simulated modem reboot in <code>tools/mutating_tools.py</code>	<code>MODEM_MGMT_BASE_URL</code> HTTP adapter

All base URLs are read from environment variables. Provide a `.env.example` to the client with the required variable names.

10. Environment Variables Summary

The client must provision the following environment variables on the deployment host:

```
# AWS / LLM
AWS_REGION=us-east-1
BEDROCK_MODEL_ID=us.anthropic.claude-haiku-4-5-20251001-v1:0

# CRM
CRM_BASE_URL=https://crm.client.internal/api/v1
CRM_CLIENT_ID=
CRM_CLIENT_SECRET=
CRM_TOKEN_URL=https://auth.client.internal/oauth2/token

# Network Diagnostics
NETWORK_BASE_URL=https://network.client.internal/api/v1
NETWORK_CLIENT_ID=
NETWORK_CLIENT_SECRET=

# Outage System
OUTAGE_BASE_URL=https://outage.client.internal/api/v1
OUTAGE_CLIENT_ID=
OUTAGE_CLIENT_SECRET=

# Modem Management
MODEM_MGMT_BASE_URL=https://nms.client.internal/api/v1
MODEM_MGMT_CLIENT_ID=
MODEM_MGMT_CLIENT_SECRET=

# KB / Documentation
KB_BASE_URL=https://kb.client.internal/api/v1
KB_CLIENT_ID=
KB_CLIENT_SECRET=

# SMS / Notifications
SMS_BASE_URL=https://notify.client.internal/api/v1
SMS_CLIENT_ID=
SMS_CLIENT_SECRET=
```